RM

伪代码

```
initialize w

repeat for each sample x_k:
    noisy_error = x_k - w
    w = w + alpha_k * noisy_error
    如果目标是估计均值 $\mathbb{E}[X]$，这一行就是：
    w = w + alpha_k * (x_k - w)
```

w 是参数 / 估计量。不是 x 要趋近 w，而是 w 要在很多样本的推动下趋近 E[X]。

$w \leftarrow w + \alpha(xk - w)$

和 MC 的区别

方法	St的更新目标	是否等 episode 结束
MC	完整回报 Gt	要等
TD	$Rt + \mathbf{1} + \gamma V(St + \mathbf{1})$	不用等

和 TD 有什么关系？

如果 xk来自同一个分布，那么 w 会慢慢稳定到这个分布的均值附近

V(s) 或 Q(s,a) 就是 w，每次采样得到的 return 或 TD target 就是 xk

所以就是 用采样结果去拟合真实的V或者Q

TD learning：

$V(St) \leftarrow V(St) + \alpha[Rt + \mathbf{1} + \gamma V(St + \mathbf{1}) - V(St)]$

TD时间差 即时

为什么st的值要用t+1状态下的值去拟合？

TD 不想等完整 episode 结束，所以它用当前已有的估计

我不知道 St后面完整真实回报是多少，但我已经对 St+1 之后的未来有一个估计了，那我就用它先更新 St

$Gt \approx Rt + \mathbf{1} + \gamma V(St + \mathbf{1})$

$V(St + \mathbf{1}) \approx Gt + \mathbf{1}$

这就叫 bootstrapping

为什么可以？贝尔曼期望方程

状态价值满足：
Vπ(s)=Eπ[Rt+1+γVπ(St+1)|St=s]

这句话的意思是：
一个状态的价值，等于当前一步奖励加上下一个状态价值的折扣期望。

TD 更新其实就是在用采样版本逼近这个公式。

完整期望是：
E[Rt+1+γV(St+1)]

但我们没有完整环境分布，所以每次只采样一次：
Rt+1+γV(St+1)

然后一点点更新。

st+1怎么获得？

当前价值表里已经存着的估计值

不要求一开始准，先用当前估计凑合更新；随着采样越来越多，所有状态的估计会互相修正，逐渐稳定

一开始是随便初始化的（可以是）

终点附近的价值先变得比较准确
↓
倒数第二个状态被修正
↓
更前面的状态被修正
↓
越来越前面的状态也慢慢变准

关键点：TD 是用"旧估计"更新"当前估计"

你可以把 TD 看成：

查表得到 V(S_{t+1})
↓
用 R + γV(S_{t+1}) 构造目标
↓
更新 V(S_t)

所以 V(St+1) 的来源是：当前价值函数里已有的估计值。

它可能来自：

初始值；
之前 episode 的更新；
之前时间步的 TD 更新；
如果是神经网络，则来自当前网络对 St+1的预测。

如果用神经网络怎么办？

如果不是表格，而是函数近似，比如：V(s;w)

那么 V(St+1) 就是把 St+1输入神经网络得到的输出：V(St+1;w)

也就是：

输入 S_{t+1}
↓
神经网络 V(s;w)
↓
输出 V(S_{t+1})

然后用它构造 TD target：

Rt+1+γV(St+1;w)

再更新当前状态的预测 V(St;w)。

V(St+1) 是从当前价值表 / 当前价值网络里查出来的一个旧估计。

TD 的特点就是：

用已有的、不完美的下一状态估计，来逐步修正当前状态估计。

所以它叫 bootstrapping：
用自己的估计作为目标的一部分。